



GUJARAT TECHNOLOGICAL UNIVERSITY

Shree Swami Atmanand Saraswati

Institute of Technology

Affiliated with GTU



A Mini Project Report on

**TALENTTRACK–JOBMARKET ANALYTICS AND
JOB RECOMMENDATION SYSTEM USINGPYTHON**

Python for Data Science (BE05000231) COMPUTER ENGINEERING.

Submitted by group:

Sr.	Name of student	Enrollment No.
1	Nidhi J.Devani	240760107035
2	Priyanshi H.Dhamat	240760107036
3	Shriya A.Dhameliya	240760107038
4	Dhruvi J.Darrji	240760107044
5	Mohi Gabani	240760107048
6	Sneha R.Hirpara	240760107070

**SHREESWAMIATMANANDSARASWATIINSTITUTE OF
TECHNOLOGY**

Opp. Kapodra police station, varachha
road, Surat – 395006



CERTIFICATE

This is to certify that the Project entitled “**TALENT TRACK JOBMARKET ANALYTICS AND JOB RECOMMENDATION SYSTEM USING PYTHON**” has been prepared by **Devani Nidhi (240760107035), Dhamat Priyanshi (240760107036), Dhameliya Shriya (240760107038), Dhruvi Darji (240760107044) Gabani Mohi (240760107048),** and **Hirapara Sneha (240760107070)** in the subject of **Python for data science** under my guidance in partial fulfillment of the degree of **Bachelor of Engineering in Computer Engineering Department (5th Semester)** of Gujarat Technological University, Ahmedabad, during the academic year **2026–2027**.

Date:

S S A S I T

Place: SSASIT, Surat

Project Guide
Prof. Vipul Kania

Head of Department
Prof. Chirag R. Patel

TABLE OF CONTENTS

Sr. No	Contents	Page no.
1	Abstract	4
2	Chapter 1: Problem Statement	5
3	Chapter 2: Objectives	6-7
4	Chapter 3: Literature Survey	8-9
5	Chapter 4: System Design	10-11
6	Chapter 5: Algorithm	15-17
7	Chapter 6: Flowchart	18-19
8	Chapter 7: Implementation	20-25
9	Chapter 8: Result and Discussion	26
10	Chapter 9: System Testing	27-28
11	Chapter 10: Result Screen/ Output Description	29-32
12	Chapter 11: Advantages of the system	33
13	Chapter 12: Limitations	34
14	Chapter 13: Conclusion	35
15	Chapter 14: Future Scope	36-37
	References	38

ABSTRACT

The Talent Track – Job Market Analytics and Recommendation System is a Python-based Data Science mini project designed to analyze information generated by a job portal. Talent Tracker Pro is a full-stack recruitment and talent management web application developed to provide a centralized platform for job seekers, employers, and administrators. The main purpose of the system is to simplify the recruitment process by bringing job searching, job posting, online applications, candidate management, and application tracking into a single platform. The system is designed using a three-layer architecture consisting of the frontend, backend, and database layers. The frontend is developed using HTML5, CSS, and JavaScript to provide an interactive and user-friendly interface. The backend is developed using Python Flask and provides REST API services for handling authentication, user profiles, job management, applications, and administrative operations. SQLite is used as the database to store information related to users, jobs, applications, and saved jobs. The system supports three major types of users: Job Seekers, Employers, and Administrators. Job seekers can create profiles, search and filter jobs, view job details, apply for suitable positions, and track their applications. Employers can post job vacancies, view applications, manage candidates, and update application status. Administrators can manage users, jobs, applications, and system statistics. Overall, TalentTracker Pro provides a structured and efficient approach to recruitment management. It reduces the need for manual processes, improves access to recruitment information, and provides a single platform for managing interactions between candidates and employers.

1. PROBLEM STATEMENT

Recruitment is an important process for both organizations and job seekers. In a traditional recruitment process, candidates search for vacancies through different sources, prepare their information separately, and communicate with employers through emails or other platforms.

Employers also have to collect applications, check candidate details, shortlist suitable candidates, and maintain recruitment records. Managing all these activities manually can take considerable time and may lead to difficulty in maintaining accurate information.

Another problem is that job information, candidate information, and application information are often stored separately. A job seeker may have to search through multiple job listings to find a suitable opportunity. After applying, the candidate may also have difficulty knowing the current status of the application.

The Talent Tracker Pro project is developed to provide a centralized web-based recruitment platform. The system allows job seekers to create profiles, search for available jobs, view job details, submit applications, and track their applications. Employers can create job postings and manage candidates who apply for their jobs. An administrator can monitor users, jobs, and applications.

The project follows a client-server architecture. The frontend provides the user interface, while the backend provides REST API services for authentication, profile management, job management, applications, and administrative operations.

The project documentation identifies HTML5, CSS, JavaScript, Python Flask, Flask-CORS, and SQLite as the main technologies used.

Therefore, the main problem addressed by this project is:

To develop a centralized, user-friendly and database-driven recruitment platform that connects job seekers and employers while providing efficient job searching, application management, candidate tracking, and administrative control.

2. OBJECTIVES

The main objective of Talent Tracker Pro is to develop a web-based recruitment platform that simplifies the interaction between job seekers and employers.

The major objectives of the project are as follows:

2.1 To provide a centralized recruitment platform

The system provides a single platform where users can search for jobs, create profiles, apply for vacancies, and manage recruitment-related information.

2.2 To provide user registration and login

The system allows users to create accounts and log in using their registered email address and password. Different user roles are supported, including job seekers, employers, and administrators.

2.3 To allow job seekers to search for jobs

Users can explore available job opportunities using different search and filtering parameters. Jobs can be searched using keywords and filtered according to job type and category.

2.4 To provide detailed job information

Each job listing contains information such as:

- Job title
- Company
- Location
- Employment type
- Job category
- Salary/CTC

This allows candidates to understand the opportunity before applying.

2.5 To allow employers to publish jobs

Employers can create job postings by providing information such as job title, company name, location, salary, employment type, category, required skills, and job description.

2.6 To provide an online application system

Job seekers can apply for jobs through the platform. They can provide a cover letter, expected salary, phone number, skills, and portfolio URL.

2.7 To provide application tracking

The system maintains application information and provides different application statuses such as:

- Pending
- Shortlisted
- Rejected
- Hired

This creates a simple applicant tracking mechanism.

2.8 To provide employer and administrator dashboards

Employers can view candidates who have applied for their jobs and update their application status. Administrators can monitor users, jobs, and applications across the platform.

2.9 To maintain persistent data

The system uses SQLite as a relational database so that user, job, and application information can be stored and retrieved systematically.

2.10 To create a simple and responsive interface

The frontend is designed using HTML, CSS, and JavaScript. The interface includes navigation, cards, forms, tables, dashboards, notifications, and responsive layouts.

3. LITERATURE SURVEY

Recruitment management systems have become an important application area of web technology. Traditional recruitment mainly depended on newspapers, employment agencies, emails, and direct communication. With the growth of web applications, recruitment activities have increasingly moved toward online platforms.

3.1 Traditional Recruitment Systems

Traditional recruitment generally involves several independent steps. An organization announces a vacancy, receives candidate information, checks applications, conducts selection procedures, and communicates the result to candidates.

A centralized digital system can reduce these problems by storing recruitment information in a structured database.

3.2 Online Job Portals

Online job portals allow employers to publish vacancies and candidates to search for employment opportunities. These systems generally provide job searching, filtering, registration, profiles, and applications.

3.3 Applicant Tracking Systems

A basic ATS normally maintains candidate records and allows recruiters to move candidates through different stages. TalentTracker implements a simple version of this concept through application statuses such as Pending, Shortlisted, Rejected, and Hired.

This provides employers with a clear method for managing candidates after applications are received.

3.4 Role-Based Recruitment Platforms

Talent Tracker follows this role-based approach through three roles:

1. Job Seeker
2. Employer
3. Administrator

The role of a user determines which operations are available to that user.

From the study of existing recruitment system concepts, the important requirements for a recruitment platform are:

- User authentication
- Application tracking
- Employer management
- Administrative management
- Database storage

TalentTracker combines these basic concepts into one academic full-stack application.

3.5 Web-Based REST Architecture

Modern web applications commonly separate the user interface from the backend services. A frontend sends requests to backend APIs, and the backend communicates with the database.

TalentTracker follows this approach. The frontend uses JavaScript Fetch API calls to communicate with Flask REST endpoints. This separation makes it possible to organize the interface and server-side operations independently.

3.6 Database-Driven Systems

Recruitment applications require persistent storage because information about users, jobs, and applications must remain available after the application is closed.

TalentTracker uses SQLite as the relational database. Separate tables are used for users, jobs, applications, and saved jobs. Relationships between these tables allow recruitment information to be connected.

3.7 Summary of Literature Survey

The study shows that modern recruitment systems require centralized information, authentication, job searching, application tracking, employer management, administrative management, and database storage. TalentTracker combines these concepts into one academic full-stack application.

4. SYSTEM DESIGN

4.1 System Overview

TalentTracker Pro is designed as a three-layer web application.

The three major layers are:

1. Presentation Layer
2. Application/Backend Layer
3. Data Layer

This design separates user-interface operations from database operations.

1. Presentation Layer

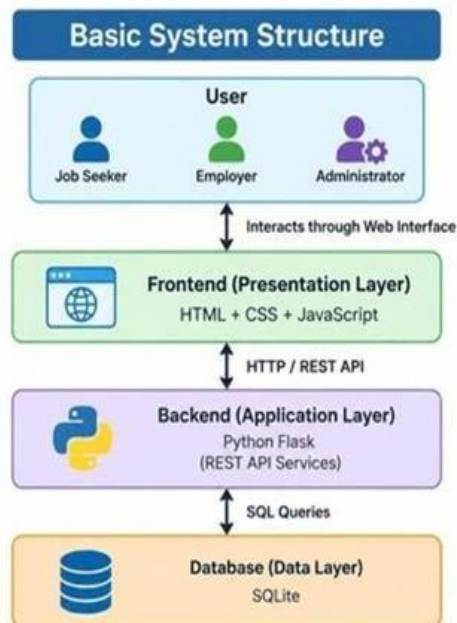
The presentation layer is responsible for displaying the user interface. It is developed using HTML, CSS, and JavaScript.

2. Application/Backend Layer

The application layer is implemented using Python Flask. It receives requests from the frontend, processes the requested operation, performs validation, communicates with the database, and returns JSON responses.

3. Data Layer

The data layer consists of the SQLite database. It stores user accounts, jobs, applications, and saved job.



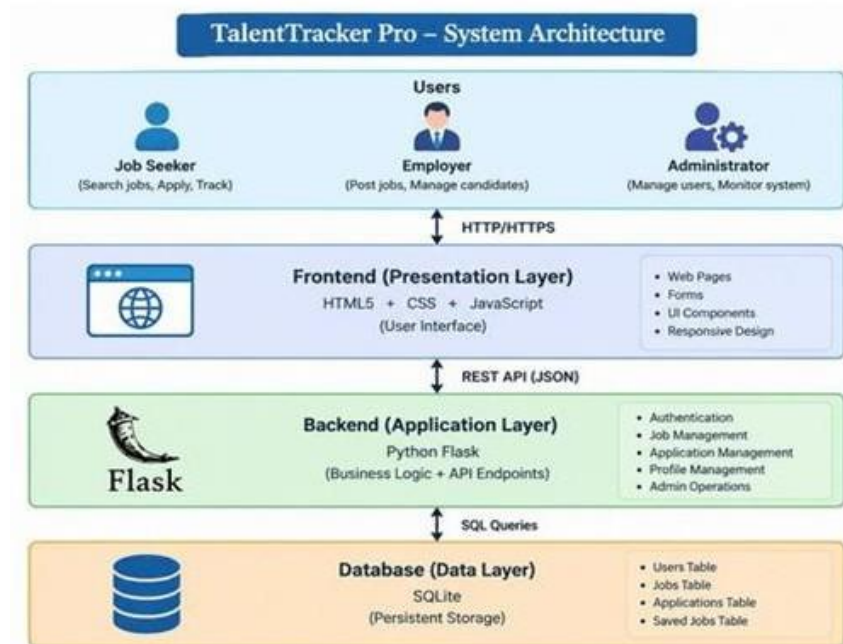
4.2 System Architecture

The architecture of TalentTracker can be represented as a client-server architecture.

The frontend provides the user interface and communicates with the Flask backend through REST APIs.

The backend processes requests and communicates with the SQLite database.

The architecture allows different users to access different functionalities while maintaining a common database.



4.3 SYSTEM MODULES

The major modules are:

Module 1: Authentication

Responsible for:

- Registration
- Login
- Session/token information
- Logout
- Current user verification

Module 2: User Profile

Allows authenticated users to maintain:

- Name
- Professional title
- Phone
- Bio
- Skills
- Portfolio

Module 3: Job Management

Provides:

- Job creation
- Job listing
- Job search
- Job filtering
- Job details
- Job deletion

Module 4: Application Management

Provides:

- Applying for jobs
- Viewing submitted applications
- Viewing applicant information
- Updating application status

Module 5: Employer Dashboard

Provides an employer with:

- Posted jobs
- Candidate applications
- Applicant details
- Application status controls

Module 6: Administrator Dashboard

Provides overall platform management, including:

- Total users
- Total jobs
- Total applications
- User management
- Job management
- Application management

4.4 DATABASE DESIGN

The database is implemented using SQLite.

The main tables are:

Users Table:

Field	Description
Id	Unique user ID
Name	User's full name
Email	Login email
Password	Account password
Role	Seeker /employer/admin
Title	Professional title
Phone	Contact number
Bio	Profile description
Skills	User skills
Portfolio	Portfolio URL
Created_at	Account creation time

Jobs Table:

Field	Description
Id	Unique job ID
Title	Job title
Company	Company name
Location	Job location
Type	Employment type
Category	Job category
salary	Salary /CTC job
Description	Description
Requirements	Required skills
Owner_id	Employer ID
Created_at	Posting time

Applications Table:

Field	Description
Id	Application ID applied
Job_id	Job
Seeker_id	Candidate
Cover_letter	Candidate introduction
Expected_salary	Expected salary
Phone	Contact number
skills	Candidate skills
Portfolio_url	Portfolio
status	Application status
Applied_at	Application time

- **Saved Jobs Table**

The project also contains a saved-jobs structure that can associate users with jobs they save for later reference.

5. ALGORITHM

5.1 User Registration Algorithm

Step 1: Start.

Step 2: Accept name, email, password, and role from the registration form.

Step 3: Check whether the required fields are empty.

Step 4: Validate the email format.

Step 5: Check whether the password satisfies the minimum length requirement.

Step 6: Check whether the selected role is valid.

Step 7: Search the database for an existing email.

Step 8: If the email already exists, display an error message.

Step 9: Otherwise, insert the new user into the users table.

Step 10: Generate the user session/token information.

Step 11: Return successful registration information.

Step 12: Stop.

5.2 User Login Algorithm

Step 1: Start.

Step 2: Accept email and password.

Step 3: Validate that both fields contain values.

Step 4: Search the users table using the email.

Step 5: If the user does not exist, return an authentication error.

Step 6: Compare the entered password with the stored password.

Step 7: If the password does not match, return an error.

Step 8: If the credentials are valid, retrieve the user information.

Step 9: Generate authentication/session information.

Step 10: Send the user information to the frontend.

Step 11: Display the appropriate dashboard.

Step 12: Stop.

5.3 Job Search Algorithm

Step 1: Start.

Step 2: Accept keyword, job type, category, and sorting option.

Step 3: Create a database query.

Step 4: If a job type is selected, filter the jobs according to the type.

Step 5: If a category is selected, filter according to category.

Step 6: If a keyword is entered, search the job title, company, location, description, and requirements.

Step 7: Apply the selected sorting method.

Step 8: Execute the SQL query.

Step 9: Convert the returned records into JSON data.

Step 10: Display the jobs on the frontend.

Step 11: Stop.

5.4 Job Application Algorithm

Step 1: Start.

Step 2: Verify that the user is logged in.

Step 3: Verify that the user has the seeker role.

Step 4: Select the required job.

Step 5: Enter cover letter, expected salary, phone, skills, and portfolio.

Step 6: Validate the submitted information.

Step 7: Check whether the candidate has already applied for the same job.

Step 8: Insert the application into the applications table.

Step 9: Set the initial application status to Pending.

Step 10: Display application success notification.

Step 11: Stop.

5.5 Application Status Algorithm

Step 1: Start.

Step 2: Employer/admin selects an application.

Step 3: Verify user authorization.

Step 4: Select the new application status.

Step 5: Check whether the status is valid.

Step 6: Update the status in the database.

Step 7: Commit the database transaction.

Step 8: Return a success message.

Step 9: Refresh the employer dashboard.

Step 10: Stop.

6. FLOWCHART

The following flowcharts can be created in Word using:

Insert → Shapes → Flowchart

6.1 Main System Flowchart

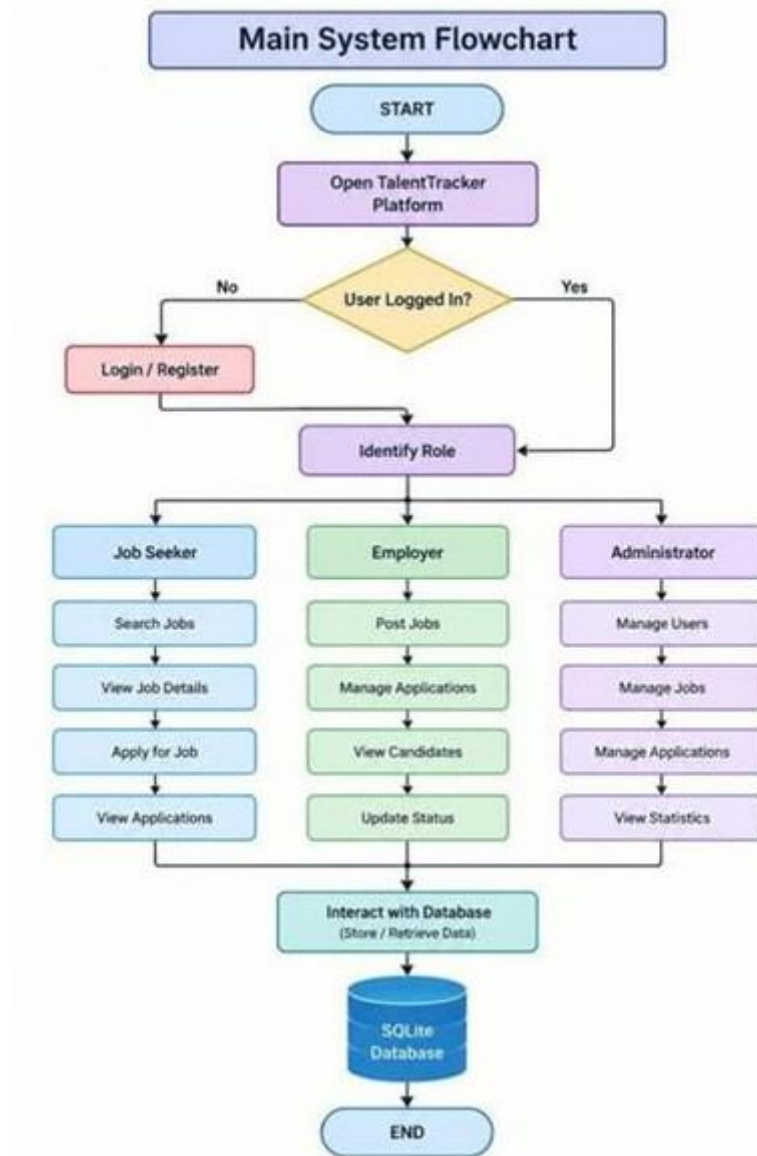


Figure 6.1: Overall System Flowchart

6.2 Login Flowchart

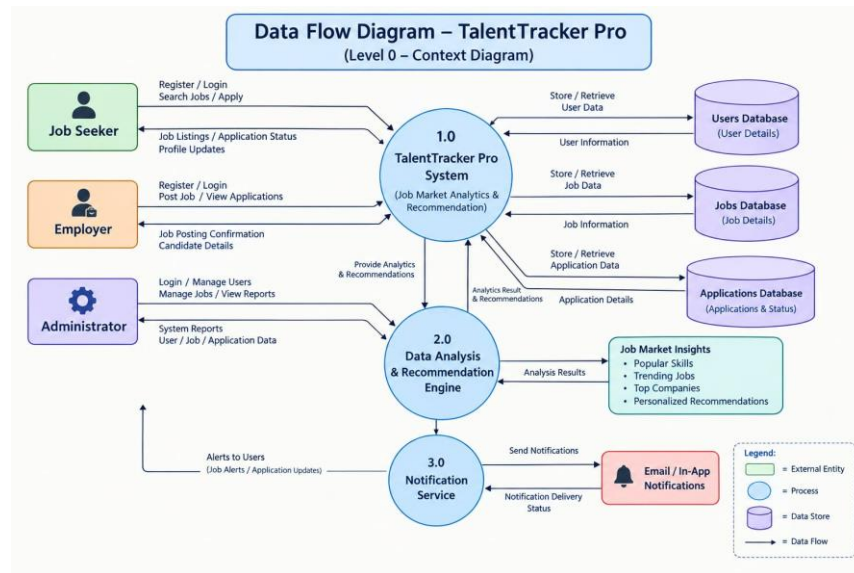


Figure 6.2: Login Flowchart

6.3 Job Application Flowchart

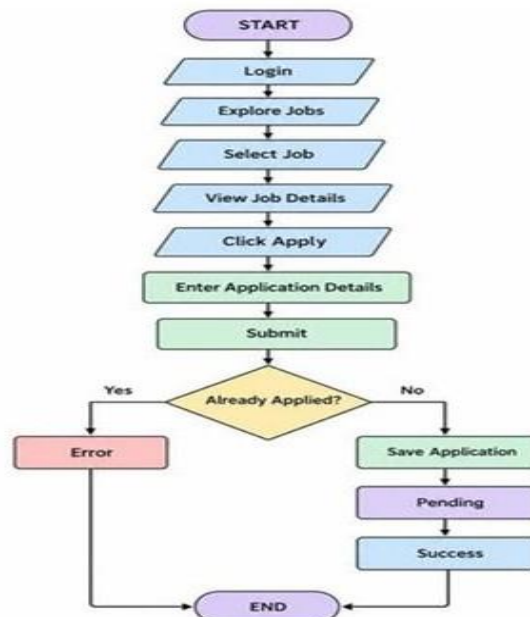


Figure 6.3: Application Flowchart

7. IMPLEMENTATION

7.1 Frontend Implementation

The frontend is implemented using **HTML5, CSS, and JavaScript**. The interface is designed as a single-page style application where different views are dynamically rendered according to the selected operation.

The navigation contains options such as:

- Home
- Explore Jobs
- Dashboard
- Profile
- Sign In
- Sign Up

The application also uses cards for job listings and statistic cards for displaying important numbers.

The frontend contains reusable UI components such as:

- Buttons
- Cards
- Forms
- Tables
- Badges
- Modals
- Notifications
- Job cards

A major advantage of this approach is that the user does not need to move between many separate HTML pages for every operation.

7.2 Backend Implementation

The backend is implemented using **Python Flask**.

Flask receives HTTP requests from the frontend and provides REST API endpoints. The frontend communicates with the backend using JavaScript Fetch API.

Important backend operations include:

- /api/register
- /api/login
- /api/me
- /api/profile
- /api/jobs
- /api/applications
- /api/admin/stats

The backend connects to SQLite using Python's SQLite functionality.

A database connection is created when required, and SQL queries are executed to retrieve or modify data.

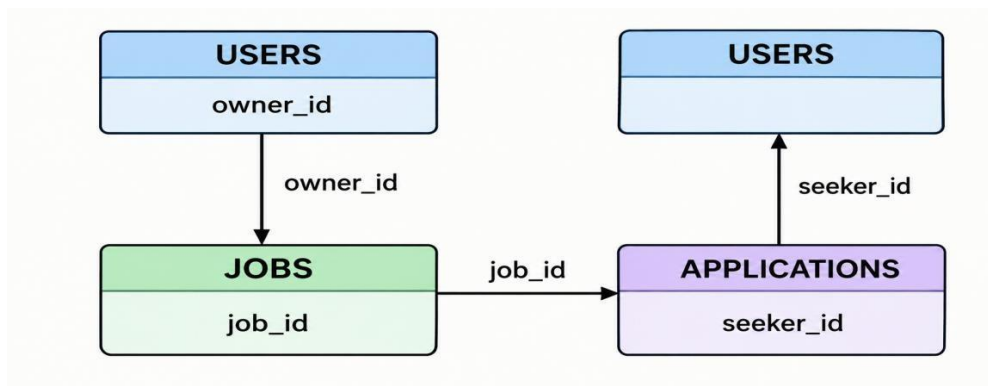
The backend returns JSON responses so that the frontend can process the results easily.

7.3 Database Implementation

SQLite is used because it is lightweight and suitable for an academic full-stack application.

The database contains separate tables for different entities.

The relationship between tables can be represented as:



This structure allows the system to identify:

- Which employer created a job
- Which candidate applied
- Which job received an application
- Which applications belong to an employer

Foreign keys are used to maintain relationships between records.

7.4 Authentication and Authorization

Authentication is used to identify the user.

During login, the backend searches the users table using the supplied email and verifies the password. After successful authentication, the frontend stores authentication information and sends it with later API requests.

The system also uses role-based authorization.

The role-based approach prevents normal users from accessing employer or administrator functions.

Job Seeker

A seeker can:

- Search jobs
- View job details
- Apply for jobs
- View applications
- Update profile

Employer

An employer can:

- Create job postings
- View applicants
- Manage applications
- Change application status
- Update profile

Administrator

An administrator can:

- View users
- View jobs
- View applications
- View platform statistics
- Manage user accounts

7.5 JOB MANAGEMENT

The job management module is one of the main components of TalentTracker.

Employers can create a new job using a form.

The job form includes:

- Job title
- Company name
- Location
- Salary package
- Employment type
- Category
- Required skills
- Job description

After submission, the information is sent to the Flask backend. The backend validates the user role and inserts the job into the jobs table.

Job seekers can then access the job through the Explore Jobs section.

The system supports keyword search. A keyword can be matched against:

- Job title
- Company
- Location
- Description
- Requirements

The system also supports filtering according to job type and category.

Sorting options allow jobs to be organized according to different criteria.

7.6 APPLICATION MANAGEMENT

The application module connects job seekers and employers.

When a candidate selects a job, the system displays the job details. If the user is not logged in, the system asks the user to sign in.

A logged-in seeker can submit:

- Cover letter
- Expected salary
- Phone number
- Skills
- Portfolio URL

After successful submission, the application is stored in the database.

The application initially receives the status:

Pending

An employer can later update it to:

- Shortlisted
- Rejected
- Hired

This provides a simple applicant tracking workflow.

7.7 DASHBOARD IMPLEMENTATION

Seeker Dashboard

The seeker dashboard displays the candidate's application information.

It can show:

- Total applications
- Job title
- Company
- Application status
- Application date

This gives the candidate a centralized view of submitted applications.

Employer Dashboard

The employer dashboard acts as a talent management center.

It provides information about candidates who have applied to the employer's jobs.

Employers can view candidate information and perform actions such as:

- Shortlist
- Hire
- Reject

The dashboard therefore provides a basic applicant tracking system.

Administrator Dashboard

The administrator dashboard provides an overall view of the platform.

It displays statistics related to:

- Users
- Jobs
- Applications
- Employers
- Job seekers

It also provides access to lists of users, jobs, and applications.

8. RESULT AND DISCUSSION

The developed TalentTracker Pro system successfully provides the major functions required for a basic online recruitment platform.

The project was tested using the provided database containing users, job records, applications, and saved-job records.

The database contains multiple categories of users including seekers, employers, and an administrator.

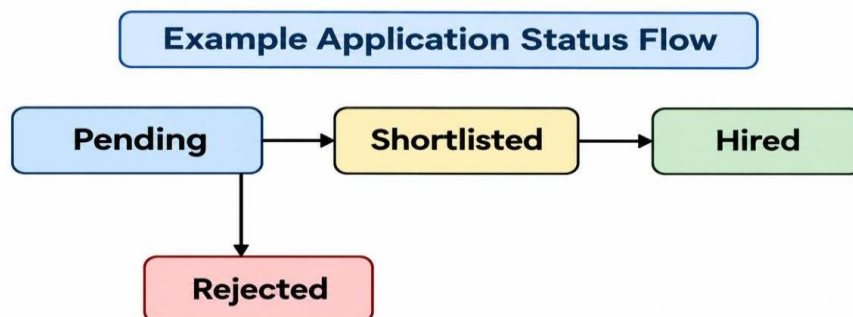
This allows role-based functionality to be tested.

The job module successfully provides job listings containing:

- Job title
- Company
- Location
- Employment type
- Category
- Salary
- Description
- Requirements

The search mechanism allows users to enter keywords and retrieve relevant job listings.

The application module provides an online method for submitting candidate information.



The dashboard provides summarized information to users according to their role.

The frontend also provides visual feedback using notifications. For example, successful operations can display success messages, while errors can display appropriate error notifications.

The system also provides a clean separation between frontend and backend functionality.

9. SYSTEM TESTING

Testing is important to verify that the different modules work correctly.

9.1 Registration Testing

Test Case: Register a new user.

Input: Valid name, email, password and role.

Expected Result: User account should be created.

Observed Result: Registration information is stored in the database and the user can continue using the platform.

9.2 Invalid Email Testing

Test Case: Enter an incorrectly formatted email.

Expected Result: System should display an email validation error.

Result: Invalid email is rejected.

9.3 Login Testing

Test Case: Login using valid credentials.

Expected Result : User should be authenticated.

Result: User is successfully authenticated and redirected to the appropriate application interface.

9.4 Job Search Testing

Test Case: Search for a keyword such as "Python".

Expected Result : Jobs related to Python should be displayed.

Result: Matching job records are retrieved through the API and displayed on the frontend.

9.5 Job Application Testing

Test Case: Apply for an available job.

Expected Result : Application should be stored with Pending status.

Result: Application is added to the application records.

9.6 Application Status Testing

Test Case: Employer changes a candidate's status to Shortlisted.

Expected Result : Application status should change.

Result: Status is updated in the database and displayed on the dashboard

10. RESULT SCREEN / OUTPUT DESCRIPTION

Figure 10.1 — Home Page

The home page introduces the TalentTracker platform and provides quick access to job searching, registration, and dashboard functionality. It also displays platform statistics and featured job opportunities.

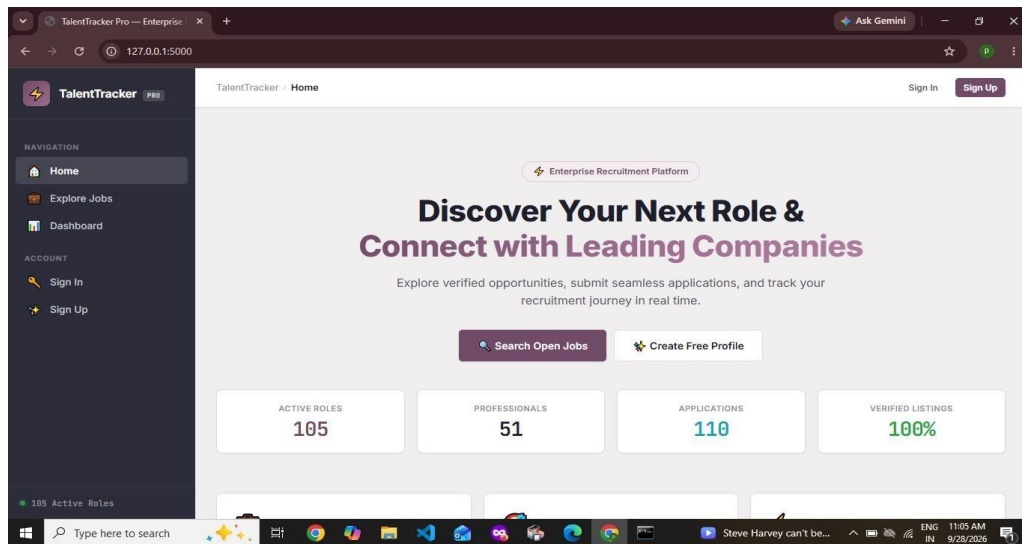


Figure 10.2 — Registration Page

The registration page allows new users to create an account by entering their personal details and selecting an appropriate role.

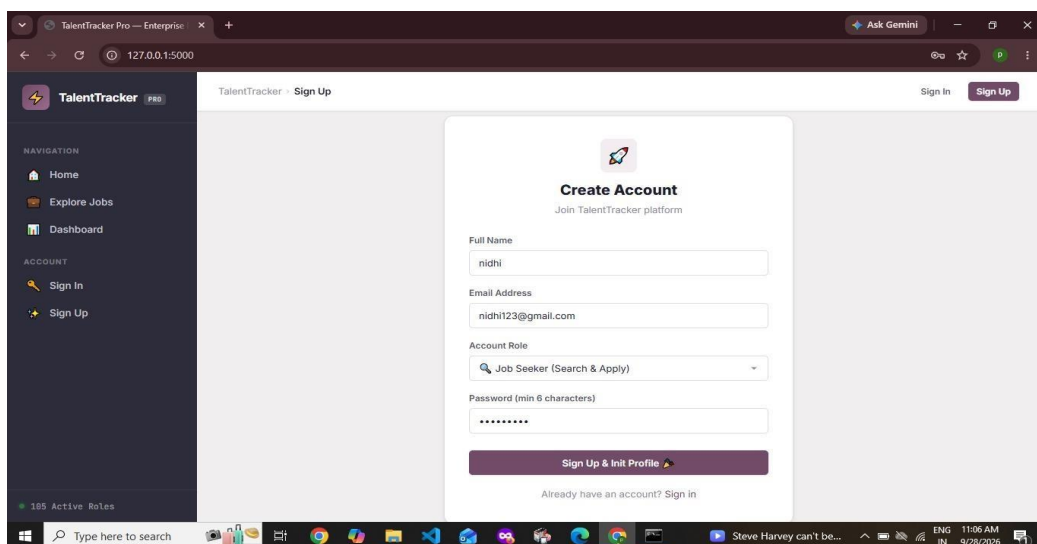


Figure 10.3 — Explore Jobs Page

The Explore Jobs page displays available job opportunities. Users can search and filter jobs according to their requirements.

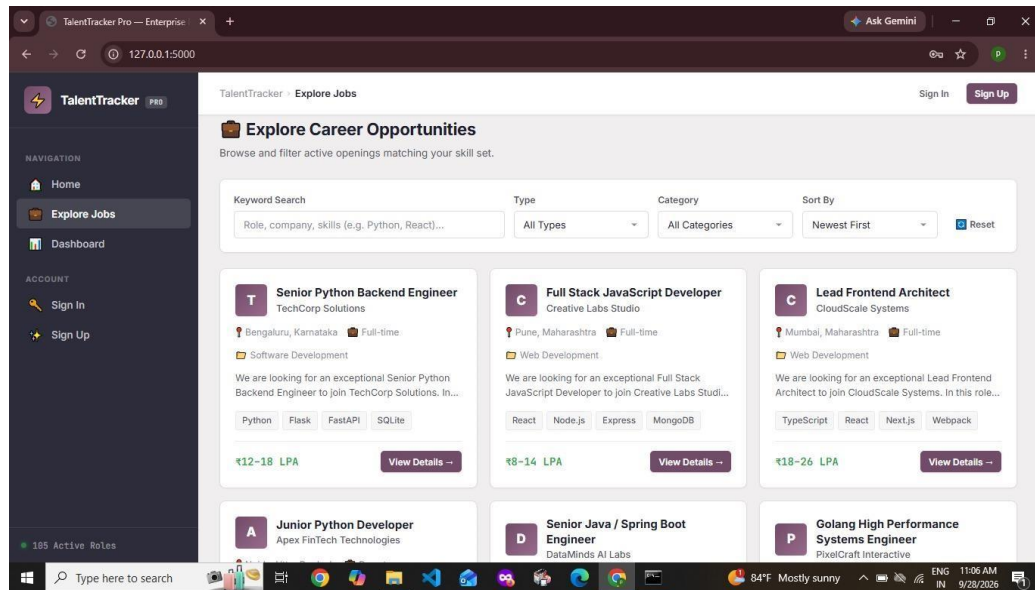


Figure 10.4— Job Details Page

The job details page displays complete information about a selected job opportunity.

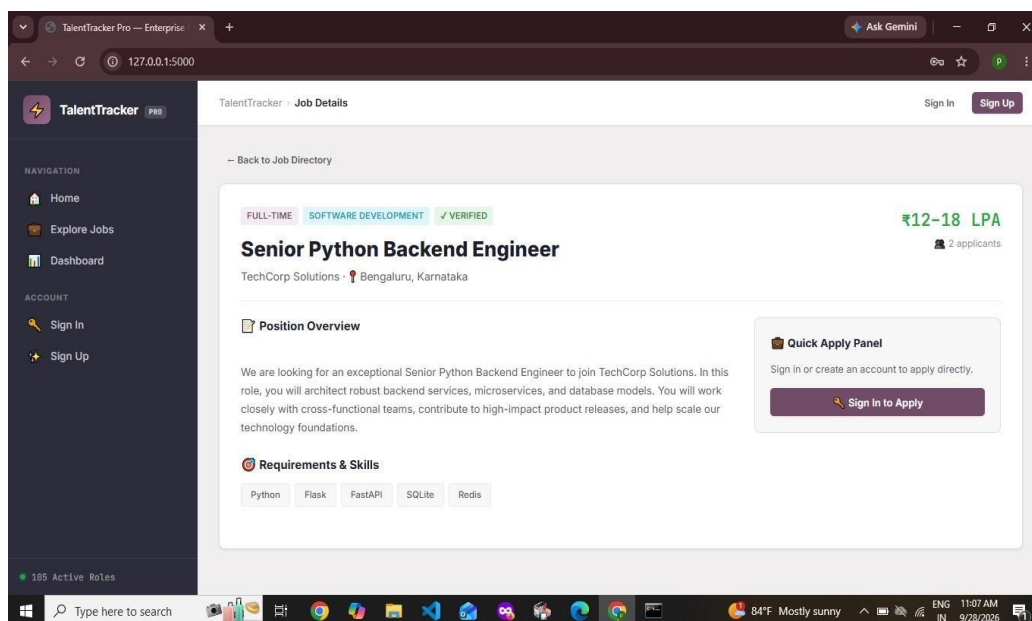
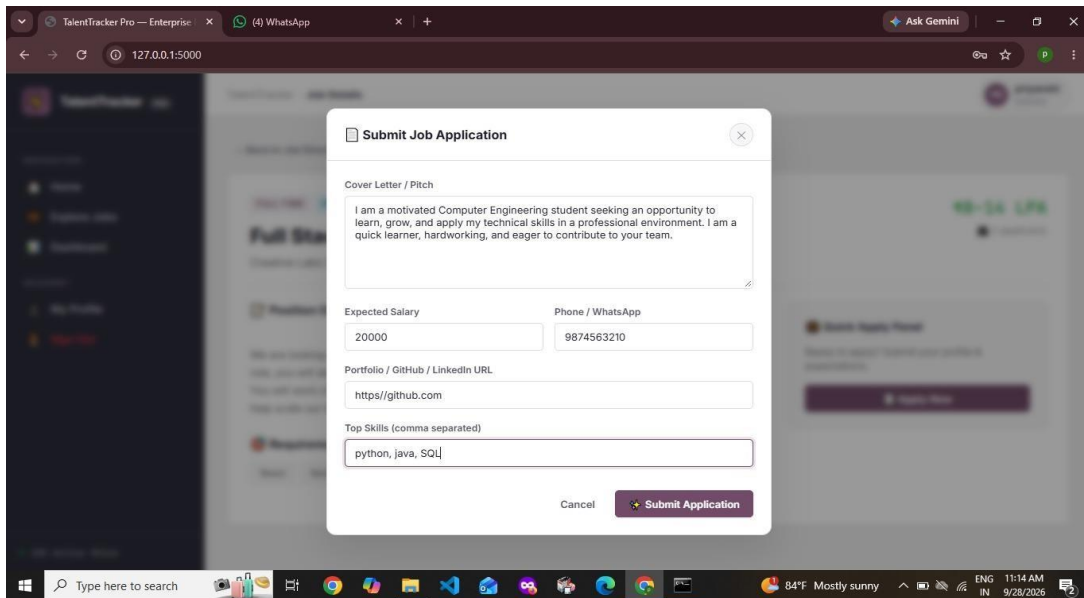


Figure 10.5 — Job Application Form

The job application form collects candidate information required to apply for a job.



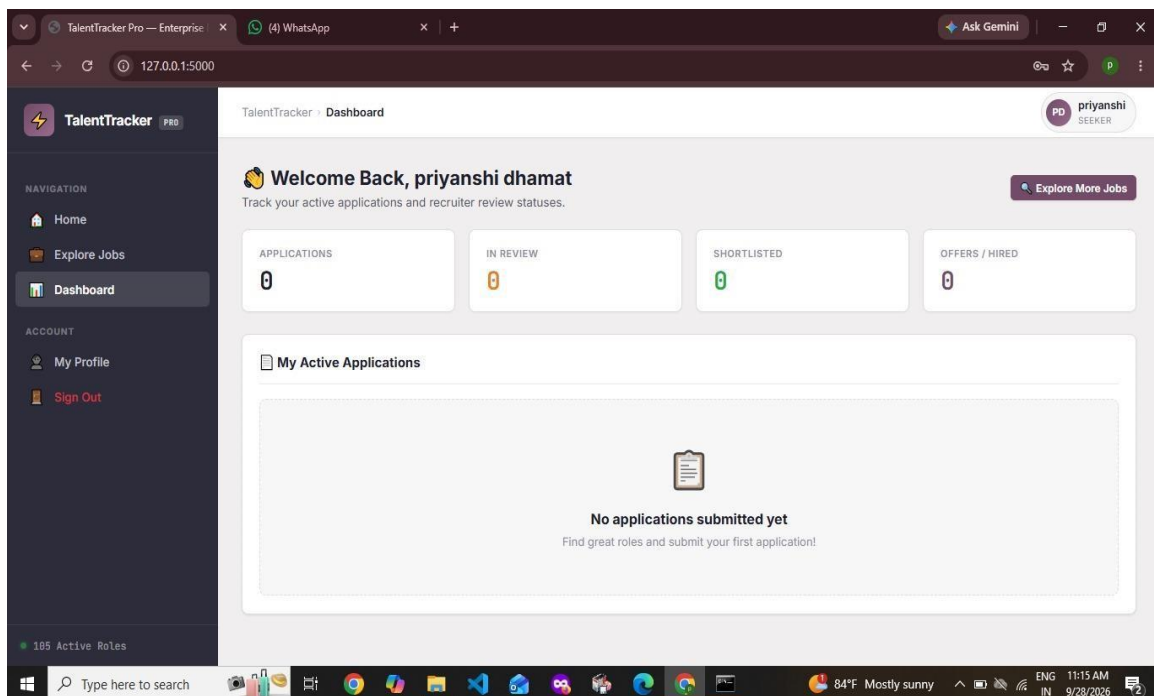
The screenshot shows a web browser window with the URL 127.0.0.1:5000. The browser has tabs for 'TalentTracker Pro — Enterprise', '(4) WhatsApp', and 'Ask Gemini'. A modal form titled 'Submit Job Application' is displayed in the center. The form contains the following fields:

- Cover Letter / Pitch:** A text area containing the text: 'I am a motivated Computer Engineering student seeking an opportunity to learn, grow, and apply my technical skills in a professional environment. I am a quick learner, hardworking, and eager to contribute to your team.'
- Expected Salary:** A text input field containing '20000'.
- Phone / WhatsApp:** A text input field containing '9874563210'.
- Portfolio / GitHub / LinkedIn URL:** A text input field containing 'https://github.com'.
- Top Skills (comma separated):** A text input field containing 'python, java, SQL'.

At the bottom of the form are two buttons: 'Cancel' and 'Submit Application'.

Figure 10.6 — Seeker Dashboard

The seeker dashboard displays information about applications submitted by the candidate.



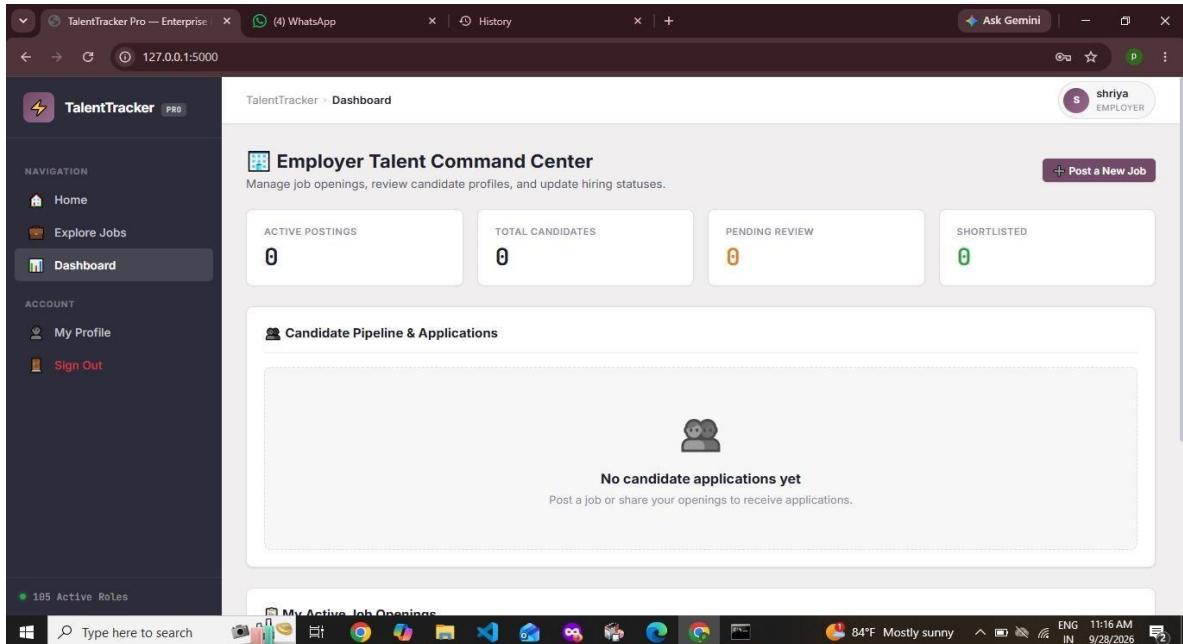
The screenshot shows the 'TalentTracker Pro' Seeker Dashboard. The user is logged in as 'priyanshi dhamat' (SEEKER). The dashboard includes a sidebar with navigation links: Home, Explore Jobs, Dashboard, My Profile, and Sign Out. The main content area displays a welcome message and a summary of the user's application status:

- APPLICATIONS:** 0
- IN REVIEW:** 0
- SHORTLISTED:** 0
- OFFERS / HIRED:** 0

Below this summary is a section titled 'My Active Applications' which shows a message: 'No applications submitted yet. Find great roles and submit your first application!'.

Figure 10.7 — Employer Dashboard

The employer dashboard displays candidate applications and provides controls for managing application status.



11. ADVANTAGES OF THE SYSTEM

Centralized Information

User, job, and application information is maintained in one system.

Reduced Manual Work

Employers can manage applications digitally instead of maintaining separate records manually.

Easy Job Search

Candidates can search jobs using keywords and filters.

Application Tracking

Candidates can view their application information, while employers can update candidate statuses.

Role-Based Access

Different users receive different functionality according to their roles.

Database Storage

Information is stored persistently using SQLite.

Responsive Interface

The frontend uses responsive CSS so that the system can adapt to different screen sizes.

Modular Design

The separation of frontend, backend, and database makes the system easier to understand and maintain.

12. LIMITATIONS

Password Security

The current academic implementation stores and compares passwords directly. A production system should use secure password hashing.

Advanced Authentication

The current authentication approach can be replaced by a stronger token system such as JWT or another secure session mechanism.

Resume Upload

The present application collects portfolio information through a URL. A future version could provide secure resume uploading.

Advanced Candidate Matching

The system does not currently provide sophisticated AI-based candidate-job matching.

Communication Features

Direct communication between employers and candidates is not a major feature of the current implementation.

Email Notifications

The system could be extended with automatic emails for application updates.

These limitations provide opportunities for future development.

13. CONCLUSION

TalentTracker Pro is a full-stack web-based recruitment and talent management platform designed to connect job seekers and employers through a centralized system.

The project successfully demonstrates how frontend technologies, backend REST APIs, and a relational database can be integrated into one application.

The frontend provides an interactive interface for:

- Registration
- Login
- Job searching
- Job details
- Applications
- Dashboards
- Profile management

The backend provides REST API endpoints for:

- Authentication
- Job management
- Application management
- Profile operations
- Administration

The project also demonstrates important concepts of modern web application development, including:

- REST APIs
- Database connectivity
- Client-server communication
- Form validation
- Dynamic content rendering
- Authentication
- Authorization
- Responsive design
- CRUD operations

14. FUTURE SCOPE

TalentTracker Pro can be further developed by adding several advanced features.

AI-Based Job Recommendation

An AI recommendation system could analyze the skills and experience of a candidate and suggest suitable jobs.

For example, if a candidate has skills in Python, Flask, SQL, and REST APIs, the system could identify jobs requiring similar skills.

AI-Based Resume Analysis

A resume parser could extract:

- Education
- Skills
- Experience
- Certifications
- Projects

from uploaded resumes.

The extracted information could automatically populate the candidate's profile.

Skill-Based Candidate Matching

Employers could receive a list of candidates whose skills closely match the requirements of a job.

Resume Upload

Candidates could upload PDF or DOCX resumes directly to the platform.

Email Notifications

The system could automatically send emails when:

- An application is submitted
- A candidate is shortlisted
- A candidate is rejected
- A candidate is hired
- A new job matching a saved preference is posted

Employer-Candidate Messaging

A secure messaging system could allow employers and candidates to communicate within the platform.

Advanced Analytics

The administrator dashboard could provide graphical reports showing:

- Number of applications
- Most active employers
- Popular job categories
- Application status distribution
- Monthly recruitment trends

Cloud Deployment

The system could be deployed on a cloud platform so that users can access it through the internet.

Improved Security

Future versions should implement:

- Password hashing
- Secure sessions
- HTTPS
- Stronger access control
- Input sanitization
- Rate limiting
- Secure API authentication

Mobile Application

A mobile application could be developed for Android and iOS, allowing candidates and employers to access the system through smartphones.

REFERENCES

1. **MDN Web Docs**
<https://developer.mozilla.org/en-US/docs/Web/HTML>
<https://developer.mozilla.org/en-US/docs/Web/CSS>
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>
2. **MDN Web Docs – Fetch API**
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
3. **Python Documentation – sqlite3**
<https://docs.python.org/3/library/sqlite3.html>
4. **Flask Documentation**
<https://flask.palletsprojects.com/>
5. **Flask-CORS Documentation**
<https://flask-cors.readthedocs.io/>
6. **SQLite Documentation**
<https://www.sqlite.org/docs.html>
7. **GeeksforGeeks – Programming & Web Development**
<https://www.geeksforgeeks.org/>